Bare Bones Programming Shield Overview Document

By: Jim Schrempp and Bob Glicksman; v1.1, 3/21/2026

NOTICE: Use of this document is subject to the terms of use described in the document "Terms_of_Use_License_and_Disclaimer" that is included in this release package. This document can be found at:

https://github.com/TeamPracticalProjects/Bare_Bones_Programming_Shield/blob/main/Terms_of_Use_License_and_Disclaimer.pdf

© 2026 Team Practical Projects, Bob Glicksman, and Jim Schrempp. All rights reserved.

TABLE OF CONTENTS.

TABLE OF CONTENTS. 1

OVERVIEW. 2

WHAT IS “BARE BONES”? 2

WHY “BARE BONES”? 4

WHAT IS A “BARE BONES PROGRAMMER”? 5

WHAT IS IN THIS PROJECT? 6

CONTENTS OF THIS REPOSITORY. 6

- Top Level. 6
- Documentation folder. 6
- Hardware folder. 7
- Software folder. 7

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

OVERVIEW.

This document provides an overview of the ***Bare Bones Programming Shield*** (BBP) project from Team Practical Projects.

The BBP leverages built-in Arduino tools that enable the user to develop code for "bare bones" AVR microcontroller chips using the Arduino IDE and a custom shield board for an Arduino Uno R3[1].

WHAT IS "BARE BONES"?

The meaning of "bare bones", in the context of this project, is the use of a microcontroller chip in a user's project where the microcontroller itself requires no additional components[2] in order to run the user's program and interact with the rest of the project's electronics. In addition to the microcontroller needing no external components to support its embedded operation, "bare bones" means that all of the microcontroller's I/O pins and internal resources are available for use in the embedded project.

There are many ways to configure and program a microcontroller. This project focuses on using the Arduino programming system, its programming language (officially called "Wiring"), and its familiar integrated development environment (IDE) in order to develop, compile, verify and upload programs to a "bare bones" microcontroller. By using the Arduino programing language ("Wiring"), the user benefits from:

- The Wiring language's ease of use and familiarity.
- The huge number of software libraries available for the Arduino environment.
- Hardware independence of the programming language.
- Ability to breadboard and develop/debug software on an easy to use Arduino board, e.g. Uno or Nano, and then transition to a "bare bones" chip for use in the user's project.

This definition of "bare bones" necessitates removing the microcontroller chip from the project and inserting it into a hardware "Programmer" in order to upload new software to the microcontroller. This process leaves all of the microcontroller's I/O pins available to the embedded project. The project's electronics should provide a socket for the microcontroller chip so that it can be removed for re-programming purposes.

---

[1] This project has been fully tested using an Arduino Uno R3. It will likely run fine on an older Uno board or a clone, as long as the Uno board uses an ATmega328P chip. It will likely *NOT* work on an Uno R4, which uses a 32 bit 3.3 volt microcontroller.

[2] Power supply decoupling capacitors excepted.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

This project supports two specific AVR microcontrollers:

- ATmega328P[3]
- ATtiny85

The dual inline package (DIP) version of these microcontrollers must be used in order to use the programmer that is part of this project.

The project may be extended to support other AVR microcontrollers; however, this project is limited to support of these two chips. The ATmega328(P) is the same microcontroller that is used on the Arduino Uno (R3 and before) board and the Arduino Nano module. These two board level microcontrollers are, by far, the most popular boards for both hobbyist projects and commercial prototypes. The ATtiny85 is a smaller (8 pin, vs. 28 pin) microcontroller that is largely compatible with the ATmega328(P) but has fewer I/O's, less memory and lacks internal hardware support for serial I/O or I2C[4].

"Bare bones" AVR microcontroller chips have many options for clock speed and other configuration parameters. The correct set of options must be configured by setting "fuses" within the chip. This is a one time process. After a chip has been configured by setting its fuses, it need not be configured again. The process for configuring these two types of microcontroller chips is documented in the "User Manual" for this project, see:

https://github.com/TeamPracticalProjects/Bare_Bones_Programming_Shield/blob/main/Documentation/Bare_Bones_Programmer_User_Manual.pdf

NOTE: the specific configuration for both the ATmega328(P) and the ATtiny85 that is used on this project is:

- Internal clock, running at 8 MHz.

"Internal clock" means that no external components (e.g. crystal and capacitors) are required to support this configuration of the microcontroller. This is in the spirit of the meaning of "bare bones". 8 MHz is the fastest clock speed that is supported by an internal clock on these microcontrollers that still allows the microcontroller to operate over its full supply voltage range. Arduino Uno and Nano boards use an external 16 MHz clock and therefore run twice as fast. However, most embedded microcontroller projects interact with humans and/or mechanical components and their programs must be slowed down accordingly (i.e., using delay() and/or millis()). Programs developed and tested on one of these boards are very likely to work just as well on a "bare bones" ATmega328(P) chip.

---

[3] The non-P version of the ATmega328 can also be supported via a simple edit to a configuration file. The terminology ATmega328(P) is used in the documentation for this project to denote this option.

[4] Arduino software libraries are available that simulate these functions, if necessary.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

WHY "BARE BONES"?

In the context of this document, "bare bones" means using an ATtiny85 (or 45 or 25) or an ATmega328(P) chip for an embedded project without needing any external components to enable the microcontroller chip run. It will not be necessary to use a board-level (Arduino Uno) or module level (Arduino Nano) product if a "bare bones" microcontroller chip is used. The advantages of using a "bare bones" chip vs a board or module level component are:

- Lower cost.
- Smaller footprint.
- Wider range of power supply voltages (e.g. when powering from batteries)
- Ability to power down the microcontroller to extremely low power consumption while retaining the program and data state.

There are a few disadvantages to using a "bare bones" chip in lieu of an Arduino Uno or Nano:

- "Bare bones" chips are configured to run at an 8 MHz clock, vs. 16 MHz for the board level microcontrollers.

- "Bare bones" chips do not have USB interface support. Therefore, they cannot be programmed using an internal bootloader[5]. "Bare bones" chips cannot directly communicate with the Arduino Serial Monitor (Serial.println(), et. el.). If the serial monitor is needed for the user's project, an "FTDI board" or similar clone can be used for this purpose. The use of an external FTDI board somewhat mitigates the advantages of the "bare bones" chip, however.

- The process of uploading a user's program to a "bare bones" microcontroller involves removing the chip from the user's project, inserting the chip in a custom-built "Programmer" device, and then flashing the code to the chip using the Programmer. The chip must then be removed from the Programmer and re-inserted into the user's project. This process is not as convenient as using the bootloader method of an Arduino board/module, but it eliminates the USB-serial hardware interface components from the user's embedded project.

Examples of projects by Team Practical Projects that use "bare bones" microcontroller chips (and would not be feasible using board or module level microcontrollers) include:

---

[5] This project uploads code to the "bare bones" chip using in-circuit serial programming (ICSP). ICSP is a hardware mechanism that uses the chip's Reset and SPI pins to configure "bare bones" microcontrollers and to load program code into the chip's program memory. The Arduino bootloader is not required for this method to work; however, the bootloader is flashed to the ATmega328(P) as part of the configuration process.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

- https://github.com/TeamPracticalProjects/Tiny_Sensor
- https://github.com/TeamPracticalProjects/Mailbox_Notifier
- https://github.com/TeamPracticalProjects/MN_Help_System

WHAT IS A “BARE BONES PROGRAMMER”?

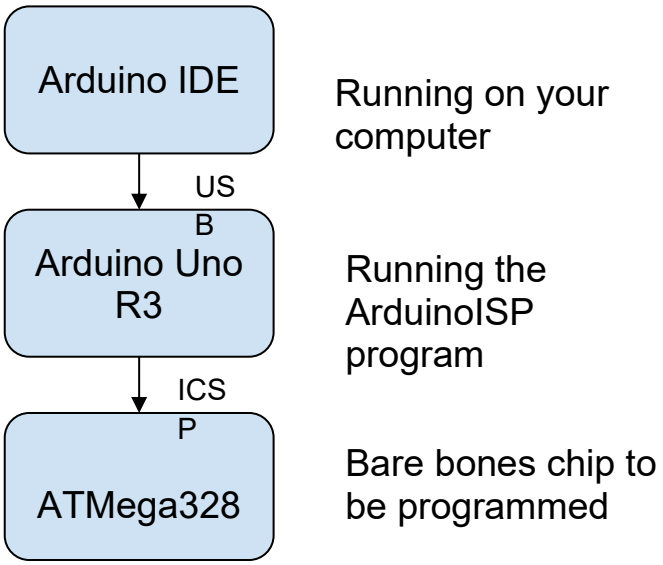
A “Bare Bones” Programmer is a device to upload programs to a “bare bones” microcontroller. This project utilizes an Arduino Uno (R3 or earlier) and a custom designed shield board to make the Programmer. The Programmer uses in-circuit serial programming (ICSP) to set fuses and flash software to the “bare bones” microcontroller. ICSP is a hardware mechanism that is built into almost every microcontroller. Once assembled, a program called “ArduinoISP” is uploaded to the Uno that is part of the Programmer. ArduinoISP is a standard program that is included in the Arduino IDE as one of the examples.

Complete instructions for building the “Bare Bones” Programmer and for adding support for ATmega328(P) and ATtiny85 “bare bones” chips are in the document “Bare_Bones_Porgrammer_and Theory_Manual” in the Documentation folder of this repository; see:

https://github.com/TeamPracticalProjects/Bare_Bones_Programming_Shield/blob/main/Documentation/Bare_Bones_Programmer_Assembly_and_Theory_Manual.pdf

Fusion 360 electronic CAD files and manufacturing (“Gerber”) files for the shield printed circuit board are in the Hardware folder in this repository; see:

https://github.com/TeamPracticalProjects/Bare_Bones_Programming_Shield/tree/main/Hardware



© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

WHAT IS IN THIS PROJECT?

This project provides all necessary files and instructions to build and utilize a “Bare Bones” Programmer that uses the Arduino IDE to compile and upload Arduino programs to a “bare bones” microcontroller. The project consists of:

- CAD and manufacturing files to manufacture a custom printed circuit Arduino Shield board.
- Documentation to understand and build the custom shield printed circuit board.
- Documentation to assemble the “Bare Bones” Programmer and to load the necessary open source software into the Programmer’s Uno.
- Documentation to add support for “bare bones” microcontroller chips to the Arduino IDE.
- Documentation to use the Programmer and IDE to configure “bare bones” microcontrollers and to compile and upload user-written software to the latter.
- A testing version of the standard Arduino Blink program that will blink an LED mounted on the Programmer’s shield board in order to test that a bare bones microcontroller is configured and runs standard Arduino software.

The custom shield board provides sockets that mount “bare bones” microcontroller chips for programming via the Arduino IDE and the attached Uno. No manual wiring is necessary. The shield board also has an LED connected to each of the programming sockets. The LED is connected to the correct pin of each type of supported microcontroller so that the LED can be controlled via Arduino digital I/O number 4. This allows the user to test a new (factory fresh) chip using the modified Arduino “blink” program while the chip is still in the programmer.

CONTENTS OF THIS REPOSITORY.

Top Level.

- *README.md*: the standard GitHub README file for this project.
- *Terms_of_Use_License_and_Disclaimer.pdf*: The open source license for this project and the requirement that you must accept in order to use the material herein.
- *Bare_Bones_Overview_Document.pdf*: This document.

Documentation folder.

- *Bare_Bones_Programmer_Assembly_and_Theory_Manual.pdf*: A manual that describes the theory of operation of the “Bare Bones” programmer and provides instructions for building one.
- *Bare_Bones_Programmer_User_Manual.pdf*: A manual that describes how to use the Programmer to configure “bare bones” microcontroller chips and how to compile and upload Arduino programs to the latter

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

Hardware folder.

- *Bare_Bones_Programming_Shield.fsch*: Fusion 360 electronic schematic file.
- *Bare_Bones_Programming_Shield.fbrd*: Fusion 360 electronic board file.
- *Bare_Bones_Programming_Shield v17_2026-02-19*: Manufacturing ("Gerber") files for the custom shield printed circuit board.

Software folder.

- Bare_Bones Blink subfolder: Arduino example blink program modified to blink an LED by a "bare bones" microcontroller while still in the Programmer.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*